

# ValidAI - Sistema de Gerenciamento e Emissão de Certificados

---

**Aluno:** Yuri Vital

## Contexto do Problema

---

**Objetivo:** Desenvolver uma API RESTful em Java (Spring Boot) para gerenciamento de eventos e emissão de certificados, focando em boas práticas, arquitetura em camadas e banco de dados relacional.

**Problema:** Instituições gerenciam inscrições, presença e emissão de certificados manualmente por planilhas e formulários básicos. Isso gera retrabalho, erros e falta de rastreabilidade.

**Solução:** Sistema de gerenciamento de eventos para instituições, no qual será possível cadastrar eventos, gerenciar inscritos, confirmar presença via QR Code e emitir certificados.

- Cadastro de usuários (participantes, organizadores e administradores).
- Criação e gerenciamento de eventos.
- Inscrição de participantes em eventos.
- Validação de presença por QR Code.
- Emissão de certificados para participantes com presença confirmada.

## Requisitos e Regras de Negócio

---

### 1. Perfis de Usuário

| Perfil       | Descrição                                                               |
|--------------|-------------------------------------------------------------------------|
| Admin        | Gerencia usuários, eventos e permissões gerais do sistema.              |
| Organizador  | Cria eventos, gerencia inscritos e valida presença.                     |
| Participante | Visualiza eventos disponíveis, realiza inscrição e recebe certificados. |

### 2. Regras de Negócio (Mínimo 10)

1. [RN01] - **User:** para se inscrever em um evento, é necessário estar cadastrado no sistema.
2. [RN02] - **User:** para se cadastrar no sistema, são obrigatórios nome, sobrenome, e-mail e senha.
3. [RN03] - **User:** só pode existir um usuário por e-mail.
4. [RN04] - **Registration:** um usuário não pode se inscrever duas vezes no mesmo evento.
5. [RN05] - **Certificate:** um participante só recebe certificado caso sua presença seja confirmada por organizador ou admin.
6. [RN06] - **Event:** apenas admins e organizadores podem criar eventos.

7. [RN07] - **Event:** título, descrição, local, número de vagas, data, hora, carga horária, palestrante e cargo/título do palestrante são obrigatórios.
8. [RN08] - **Event:** um evento deve ser cadastrado para, no mínimo, o dia seguinte ao cadastro.
9. [RN09] - **Event:** evento cancelado não permite novas inscrições. A notificação de inscritos é uma evolução futura, pois não há serviço de e-mail nesta entrega.
10. [RN10] - **Event:** evento só pode ser editado ou cancelado pelo organizador que o criou ou por admin.
11. [RN11] - **Registration:** inscrições ativas não podem exceder a capacidade do evento.
12. [RN12] - **Certificate:** a data de emissão é a data em que o certificado é emitido pelo sistema.
13. [RN13] - **Registration:** usuário pode cancelar somente a própria inscrição e apenas antes de o evento ocorrer.

### 3. Entidades do Sistema

| Entidade      | Campos principais                                                                                                | Restrições relevantes                                                                                        |
|---------------|------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------|
| users         | id, firstName, lastName, email, passwordHash, role, createdAt                                                    | email único; senha armazenada somente como hash; role em ADMIN, ORGANIZER, PARTICIPANT.                      |
| events        | id, organizerId, title, description, location, date, hour, workload, capacity, speakerName, speakerTitle, status | organizerId é FK para users; capacidade e carga horária são positivas; status em ACTIVE, CANCELED, FINISHED. |
| registrations | id, userId, eventId, registrationAt, status, attendanceConfirmed, attendanceConfirmedAt, qrCode                  | FKs para users e events; UNIQUE(userId, eventId); qrCode único; status em ACTIVE, CANCELED, ATTENDED.        |
| certificates  | id, registrationId, issueDate, certificateCode, pdfUrl                                                           | registrationId único e FK para registrations; certificateCode e pdfUrl únicos.                               |

Cardinalidades: um usuário possui muitas inscrições; um evento possui muitas inscrições; uma inscrição possui zero ou um certificado; um usuário organiza muitos eventos.

**Justificativa:** essas entidades cobrem o ciclo completo: usuários se cadastram, eventos são criados, participantes se inscrevem, organizadores validam presenças e certificados são emitidos com base nas inscrições.

### Requisitos Funcionais

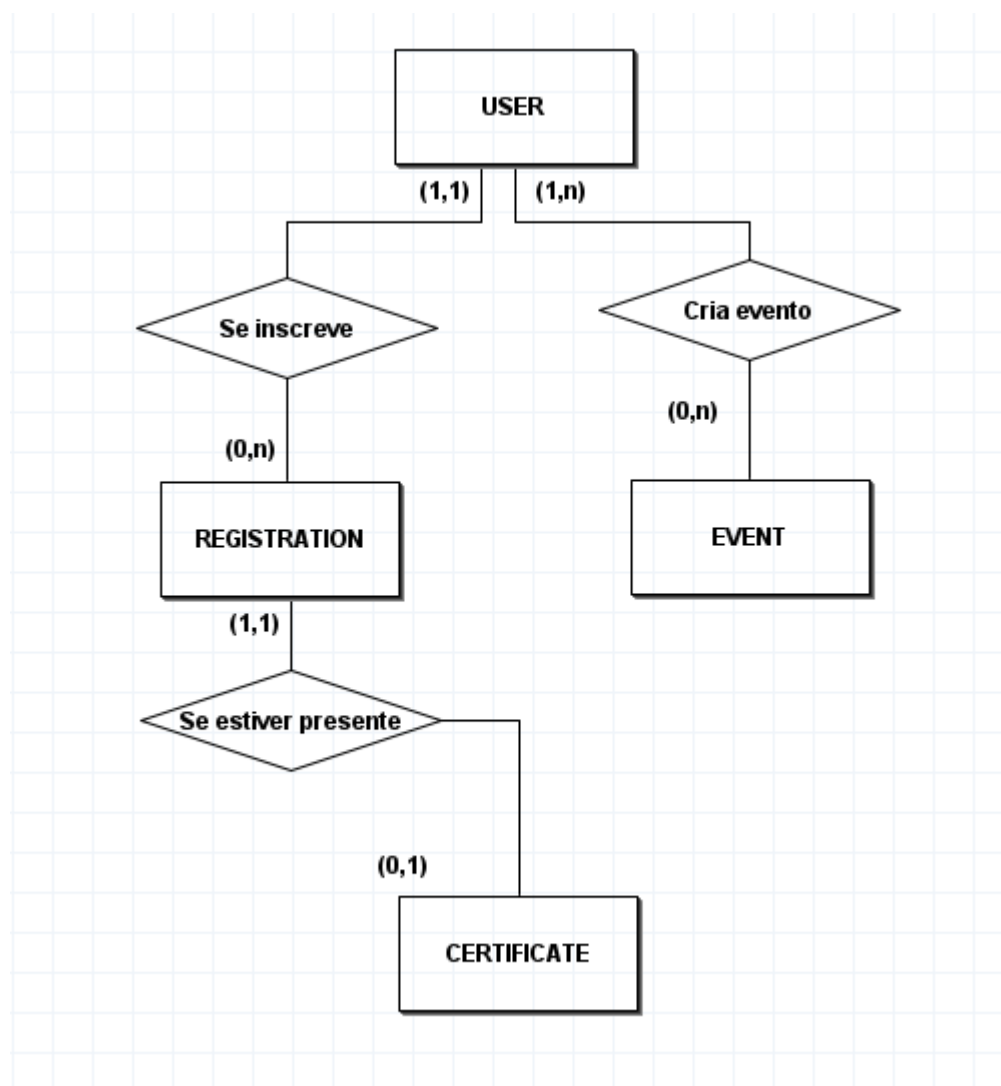
1. [RF01] Um usuário deve conseguir se cadastrar.
2. [RF02] Um organizador ou admin deve conseguir cadastrar um evento.
3. [RF03] O sistema deve confirmar a presença de participante validada via QR Code.
4. [RF04] Um usuário deve conseguir listar eventos e filtrar por status.
5. [RF05] Um usuário autenticado deve conseguir se inscrever em evento ativo com vagas disponíveis.
6. [RF06] Um participante deve conseguir cancelar sua própria inscrição antes do evento.
7. [RF07] Organizador proprietário ou admin deve conseguir editar e cancelar evento.

8. [RF08] O sistema deve gerar um token QR Code único ao criar uma inscrição.
9. [RF09] O sistema deve emitir um certificado único para inscrição com presença confirmada.
10. [RF10] Um usuário autorizado deve conseguir baixar o PDF de um certificado.
11. [RF11] Um admin deve conseguir listar usuários e alterar seus papéis.

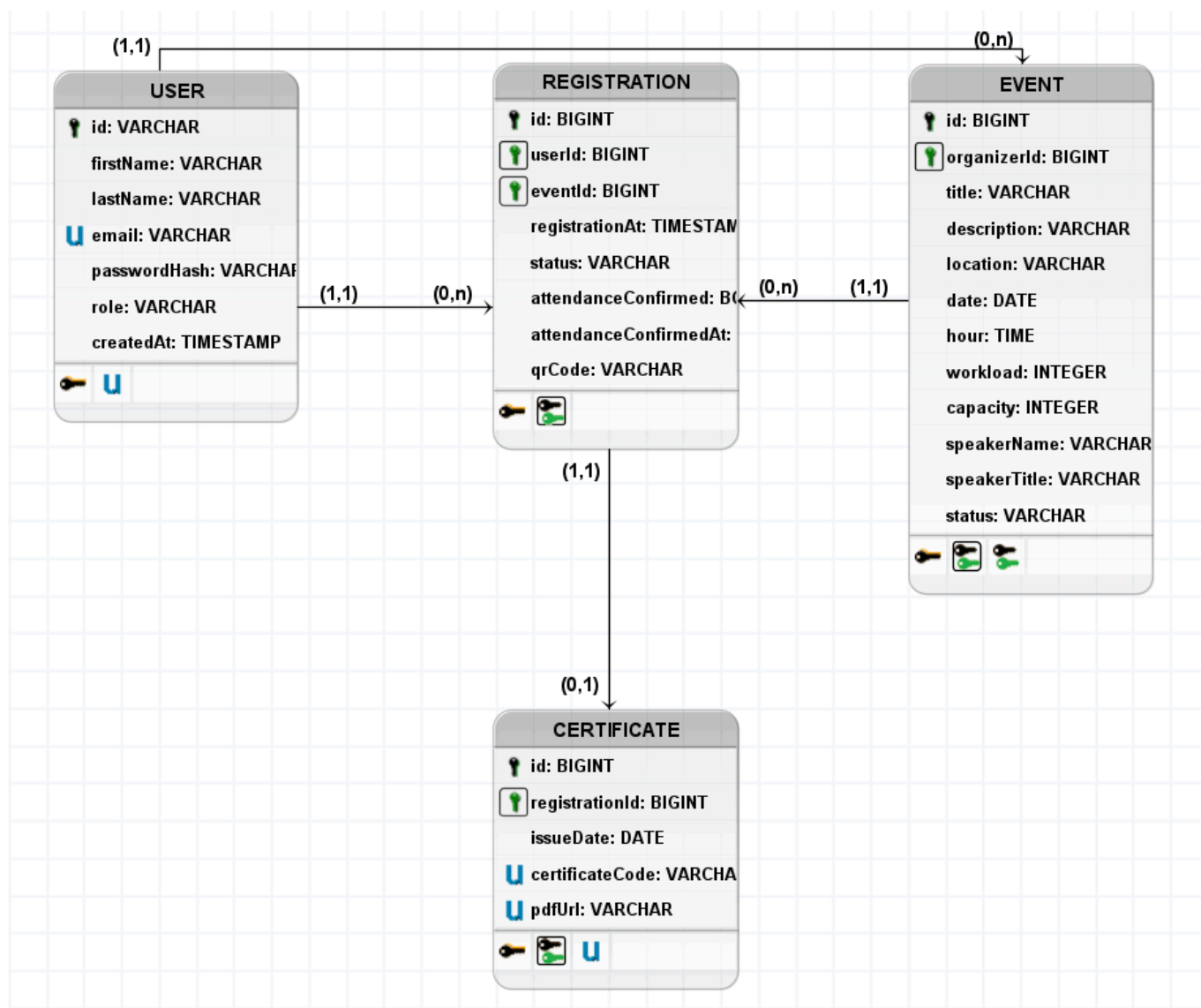
## Requisitos Não Funcionais

1. [RNF01] A API deve usar Java 21, Spring Boot 4.1.0, Gradle e PostgreSQL 16.
2. [RNF02] Entradas devem ser validadas e erros devem seguir um formato JSON padronizado.
3. [RNF03] O banco deve ser criado e evoluído por migrations Flyway versionadas.
4. [RNF04] Senhas devem ser persistidas somente como hash; JWT será implementado na unidade seguinte.
5. [RNF05] A API deve expor documentação OpenAPI/Swagger para endpoints implementados.
6. [RNF06] Regras de negócio devem ficar em services; controllers devem apenas tratar HTTP.
7. [RNF07] Tokens de QR Code e códigos de certificado devem ser únicos e não sequenciais.

## Modelagem Conceitual



## Modelagem Lógica - DER



## Dicionário de Dados

| Entidade | Campo        | Tipo         | Obrigatório | Regra                                  |
|----------|--------------|--------------|-------------|----------------------------------------|
| USER     | id           | BIGINT       | SIM         | PK                                     |
| USER     | firstName    | VARCHAR(50)  | SIM         | Mínimo 3 caracteres                    |
| USER     | lastName     | VARCHAR(50)  | SIM         | Mínimo 3 caracteres                    |
| USER     | email        | VARCHAR(255) | SIM         | UNIQUE; formato de e-mail válido       |
| USER     | passwordHash | VARCHAR(255) | SIM         | Armazenar somente hash BCrypt          |
| USER     | role         | VARCHAR(12)  | SIM         | Valores: PARTICIPANT, ORGANIZER, ADMIN |
| USER     | createdAt    | TIMESTAMP    | SIM         | Data e hora do cadastro                |
| EVENT    | id           | BIGINT       | SIM         | PK                                     |
| EVENT    | organizerId  | BIGINT       | SIM         | FK para USER(id)                       |

| Entidade     | Campo                 | Tipo                | Obrigatório | Regra                                                      |
|--------------|-----------------------|---------------------|-------------|------------------------------------------------------------|
| EVENT        | title                 | VARCHAR(255)        | SIM         | Mínimo 5 caracteres                                        |
| EVENT        | description           | VARCHAR(255)        | SIM         | Mínimo 10 caracteres                                       |
| EVENT        | location              | VARCHAR(255)        | SIM         | Mínimo 10 caracteres                                       |
| EVENT        | date                  | DATE                | SIM         | A partir do dia seguinte                                   |
| EVENT        | hour                  | TIME                | SIM         | Horário do evento                                          |
| EVENT        | workload              | INTEGER             | SIM         | Inteiro positivo; carga horária em horas                   |
| EVENT        | capacity              | INTEGER             | SIM         | Inteiro positivo; quantidade máxima de participantes       |
| EVENT        | speakerName           | VARCHAR(50)         | SIM         | Mínimo 3 caracteres                                        |
| EVENT        | speakerTitle          | VARCHAR(50)         | SIM         | Mínimo 3 caracteres                                        |
| EVENT        | status                | VARCHAR(8)          | SIM         | Valores: <b>ACTIVE</b> , <b>CANCELED</b> , <b>FINISHED</b> |
| REGISTRATION | id                    | BIGINT              | SIM         | PK                                                         |
| REGISTRATION | userId                | BIGINT              | SIM         | FK para USER(id)                                           |
| REGISTRATION | eventId               | BIGINT              | SIM         | FK para EVENT(id)                                          |
| REGISTRATION | (userId, eventId)     | COMPOSITE<br>UNIQUE | SIM         | Impede duas inscrições do mesmo usuário no evento          |
| REGISTRATION | registrationAt        | TIMESTAMP           | SIM         | Data e hora da inscrição                                   |
| REGISTRATION | status                | VARCHAR(8)          | SIM         | Valores: <b>ACTIVE</b> , <b>CANCELED</b> , <b>ATTENDED</b> |
| REGISTRATION | attendanceConfirmed   | BOOLEAN             | SIM         | <b>false</b> na criação; <b>true</b> após check-in         |
| REGISTRATION | attendanceConfirmedAt | TIMESTAMP           | NÃO         | Data e hora da confirmação de presença                     |
| REGISTRATION | qrCode                | VARCHAR(255)        | SIM         | UNIQUE; token opaco aleatório da inscrição                 |
| CERTIFICATE  | id                    | BIGINT              | SIM         | PK                                                         |
| CERTIFICATE  | registrationId        | BIGINT              | SIM         | FK para REGISTRATION(id);<br>UNIQUE (relação 1:1)          |
| CERTIFICATE  | issueDate             | DATE                | SIM         | Data atual da emissão                                      |

| Entidade    | Campo           | Tipo         | Obrigatório | Regra                                      |
|-------------|-----------------|--------------|-------------|--------------------------------------------|
| CERTIFICATE | certificateCode | VARCHAR(255) | SIM         | UNIQUE; código de validação do certificado |
| CERTIFICATE | pdfUrl          | VARCHAR(255) | SIM         | UNIQUE; caminho/URL do PDF gerado          |

## Matriz de Permissões

| Operação                                  | PARTICIPANT | ORGANIZER | ADMIN |
|-------------------------------------------|-------------|-----------|-------|
| Listar eventos                            | ✓           | ✓         | ✓     |
| Criar evento                              | —           | ✓         | ✓     |
| Editar/cancelar evento próprio            | —           | ✓         | ✓     |
| Editar/cancelar evento de terceiro        | —           | —         | ✓     |
| Inscriver-se e cancelar inscrição própria | ✓           | ✓         | ✓     |
| Consultar inscrições próprias             | ✓           | ✓         | ✓     |
| Confirmar presença em evento próprio      | —           | ✓         | ✓     |
| Emitir certificado de evento próprio      | —           | ✓         | ✓     |
| Baixar certificado próprio                | ✓           | ✓         | ✓     |
| Listar usuários e alterar papéis          | —           | —         | ✓     |

**Observação:** a matriz define o comportamento definitivo. A proteção com Spring Security/JWT será implementada após a 1AV.

## Backlog e Critérios de Aceite

| ID   | História de usuário                                        | Critério de aceite                                                                         |
|------|------------------------------------------------------------|--------------------------------------------------------------------------------------------|
| US01 | Como visitante, quero me cadastrar para acessar o sistema. | Dados válidos criam participante; e-mail repetido retorna conflito; senha não é devolvida. |
| US02 | Como usuário, quero realizar login.                        | Credenciais inválidas retornam 401; JWT será entregue na próxima unidade.                  |
| US03 | Como participante, quero listar eventos.                   | Lista pode ser vazia; filtro válido funciona; filtro inválido retorna 400.                 |
| US04 | Como organizador, quero criar e editar evento.             | Campos obrigatórios e data futura são validados; apenas proprietário/admin altera.         |
| US05 | Como organizador, quero cancelar evento.                   | Status muda para <b>CANCELED</b> e novas inscrições são bloqueadas.                        |

| ID   | História de usuário                                | Critério de aceite                                                                        |
|------|----------------------------------------------------|-------------------------------------------------------------------------------------------|
| US06 | Como participante, quero me inscrever.             | Inscrição gera token QR; duplicidade retorna 409; capacidade é respeitada.                |
| US07 | Como participante, quero cancelar inscrição.       | Somente o titular cancela antes do evento e o status muda para <b>CANCELED</b> .          |
| US08 | Como organizador, quero confirmar presença por QR. | Token de inscrição ativa do evento muda status para <b>ATTENDED</b> e registra data/hora. |
| US09 | Como autorizado, quero emitir certificado.         | Apenas inscrição atendida recebe um certificado; segunda emissão retorna 409.             |
| US10 | Como admin, quero administrar usuários.            | Admin lista usuários e altera papel válido; usuário inexistente retorna 404.              |

## Entrega Final da 1AV

Conforme **instruct.md**, a apresentação de 05/10/2026 deve demonstrar:

| Item obrigatório                       | Peso | Situação documental atual                     |
|----------------------------------------|------|-----------------------------------------------|
| Requisitos funcionais e não funcionais | 1,0  | Concluído neste documento                     |
| Regras de negócio                      | 1,0  | Concluído neste documento                     |
| DER e dicionário de dados              | 1,5  | Concluído neste documento e em <b>assets/</b> |
| Rotas, contratos e status codes        | 1,5  | Consolidado em <b>contratos-api.md</b>        |
| Arquitetura e estrutura do projeto     | 1,0  | Consolidado em <b>arquitetura.md</b>          |
| Matriz de permissões                   | 1,0  | Concluída neste documento                     |
| Backlog e critérios de aceite          | 1,0  | Concluído neste documento                     |
| Swagger/API inicial funcionando        | 1,0  | Pendente para Checkpoint 3                    |
| Organização no GitHub                  | 1,0  | Já existente; manter atualizada               |
| Apresentação e defesa técnica          | 1,0  | Pendente; preparar após MVP técnico           |

## Contratos da API - ValidAI

Este documento define todos os endpoints da API REST do sistema **ValidAI**, incluindo métodos HTTP, caminhos, exemplos de entrada e saída (JSON) e status codes possíveis. Os contratos estão alinhados ao dicionário de dados e às regras de negócio.

Todos os endpoints seguem o prefixo **/api/v1/**. A autenticação será via JWT, exceto **/users/register** e **/auth/login**; JWT será implementado na próxima unidade.

### 1.1 Cadastro de Usuário

## POST /api/v1/users/register

- **Descrição:** Cria novo usuário com papel padrão **PARTICIPANT**. Apenas administradores poderão alterar papéis posteriormente.

### Entrada (JSON)

```
{
  "firstName": "Jucelio",
  "lastName": "Testador",
  "email": "juceliotestador@email.com",
  "password": "Abcd!123"
}
```

### Saída (JSON): 201 Created

```
{
  "id": 1,
  "firstName": "Jucelio",
  "lastName": "Testador",
  "email": "juceliotestador@email.com",
  "role": "PARTICIPANT",
  "createdAt": "2026-09-01T10:00:00"
}
```

| Código | Significado                                               |
|--------|-----------------------------------------------------------|
| 201    | Usuário criado com sucesso.                               |
| 400    | Dados inválidos, como e-mail malformatado ou senha fraca. |
| 409    | E-mail já cadastrado (violação de unicidade).             |

## 1.2 Login (Autenticação)

### POST /api/v1/auth/login

- **Descrição:** Autentica o usuário e, na fase de segurança, retorna token JWT para acesso aos demais endpoints.

### Entrada (JSON)

```
{
  "email": "juceliotestador@email.com",
  "password": "Abcd!123"
}
```

**Saída (JSON): 200 OK — contrato da fase JWT**

```
{
  "accessToken": "eyJhbGciOiJIUzI1NiJ9...",
  "tokenType": "Bearer",
  "expiresIn": 3600,
  "id": 1,
  "email": "juceiotestador@email.com",
  "role": "PARTICIPANT"
}
```

| Código | Significado                                |
|--------|--------------------------------------------|
| 200    | Login bem-sucedido.                        |
| 400    | Campos obrigatórios ausentes ou inválidos. |
| 401    | Credenciais inválidas.                     |

## 1.3 Listagem de Eventos

**GET /api/v1/events**

- **Descrição:** Retorna todos os eventos cadastrados. Aceita parâmetros opcionais de filtro via query string.
- **Parâmetro opcional:** `status=ACTIVE`, `CANCELED` ou `FINISHED`.

**Entrada:** não possui corpo.

**Saída (JSON): 200 OK**

```
[
  {
    "id": 1,
    "title": "QA na prática",
    "description": "Aprenda o que é e o que faz um QA na prática",
    "location": "Teatro Algibeira",
    "date": "2026-10-10",
    "hour": "08:30",
    "workload": 20,
    "capacity": 50,
    "speakerName": "Victor Oliveira",
    "speakerTitle": "Quality Assurance Specialist",
    "status": "ACTIVE",
    "organizerId": 2
  }
]
```

| Código | Significado                                  |
|--------|----------------------------------------------|
| 200    | Lista retornada com sucesso; pode ser vazia. |
| 400    | Parâmetro de filtro inválido.                |

## 1.4 Criação de Evento

POST /api/v1/events

- **Descrição:** Cria novo evento. Apenas usuários com papel **ORGANIZER** ou **ADMIN** podem acessar. O **organizerId** é extraído do usuário logado.

### Entrada (JSON)

```
{
  "title": "Aprenda GIT de uma vez por todas!",
  "description": "Workshop com hands-on para prática ativa de versionamento com GIT",
  "location": "Teatro Algibeira",
  "date": "2026-10-16",
  "hour": "14:30",
  "workload": 10,
  "capacity": 50,
  "speakerName": "Victor Oliveira",
  "speakerTitle": "Quality Assurance Specialist"
}
```

### Saída (JSON): 201 Created

```
{
  "id": 6,
  "title": "Aprenda GIT de uma vez por todas!",
  "description": "Workshop com hands-on para prática ativa de versionamento com GIT",
  "location": "Teatro Algibeira",
  "date": "2026-10-16",
  "hour": "14:30",
  "workload": 10,
  "capacity": 50,
  "speakerName": "Victor Oliveira",
  "speakerTitle": "Quality Assurance Specialist",
  "status": "ACTIVE",
  "organizerId": 2
}
```

| Código | Significado                                                       |
|--------|-------------------------------------------------------------------|
| 201    | Evento criado com sucesso.                                        |
| 400    | Dados inválidos, como capacidade não positiva ou campo vazio.     |
| 401    | Usuário não autenticado.                                          |
| 403    | Usuário não tem permissão.                                        |
| 422    | Violação de regra de negócio, como data anterior ao dia seguinte. |

## 1.5 Inscrição em Evento

**POST** /api/v1/events/{eventId}/registrations

- **Descrição:** Inscreve o usuário autenticado em evento específico. O **eventId** é passado na URL.
- **Autenticação:** requer token JWT.

**Entrada:** nenhum corpo; o usuário é identificado pelo token JWT.

**Saída (JSON):** **201 Created**

```
{
  "id": 10,
  "userId": 1,
  "eventId": 1,
  "registrationAt": "2026-09-01T10:30:00",
  "status": "ACTIVE",
  "attendanceConfirmed": false,
  "attendanceConfirmedAt": null,
  "qrCode": "uFPyRk87C5BS1UqXgBxK0mG6..."
}
```

| Código | Significado                                             |
|--------|---------------------------------------------------------|
| 201    | Inscrição realizada e token QR gerado com sucesso.      |
| 401    | Usuário não autenticado.                                |
| 404    | Evento não encontrado.                                  |
| 409    | Usuário já inscrito no evento.                          |
| 422    | Evento lotado, cancelado ou regra de inscrição violada. |

## 1.6 Validação de Presença (via QR Code)

**PATCH** /api/v1/events/{eventId}/registrations/confirm-attendance

- **Descrição:** Organizador proprietário do evento ou admin confirma presença a partir do conteúdo lido no QR Code. A inscrição passa para **ATTENDED** e registra data/hora.

- **Autenticação:** requer token JWT.

#### Entrada (JSON)

```
{
  "qrCode": "uFPyRk87C5BS1UqXgBxK0mG6..."
}
```

#### Saída (JSON): 200 OK

```
{
  "id": 10,
  "userId": 1,
  "eventId": 1,
  "status": "ATTENDED",
  "attendanceConfirmed": true,
  "attendanceConfirmedAt": "2026-09-10T15:30:00"
}
```

| Código | Significado                                           |
|--------|-------------------------------------------------------|
| 200    | Presença confirmada com sucesso.                      |
| 400    | Corpo da requisição inválido.                         |
| 401    | Usuário não autenticado.                              |
| 403    | Usuário não é organizador do evento nem admin.        |
| 404    | Evento, QR Code ou inscrição não encontrado.          |
| 409    | Presença já confirmada.                               |
| 422    | Evento ainda não ocorreu ou inscrição não está ativa. |

## 1.7 Emissão de Certificado

### POST /api/v1/certificates

- **Descrição:** Gera certificado para inscrição que já teve presença confirmada. O sistema cria registro com código único e URL/caminho para download do PDF.
- **Autenticação:** requer token JWT; permitido ao titular, organizador do evento ou admin.

#### Entrada (JSON)

```
{
  "registrationId": 10
}
```

**Saída (JSON): 201 Created**

```
{
  "id": 5,
  "registrationId": 10,
  "issueDate": "2026-09-10",
  "certificateCode": "CERT-ABC123",
  "pdfUrl": "/api/v1/certificates/5/download"
}
```

| Código | Significado                                  |
|--------|----------------------------------------------|
| 201    | Certificado gerado com sucesso.              |
| 401    | Usuário não autenticado.                     |
| 403    | Usuário não tem permissão sobre a inscrição. |
| 404    | Inscrição não encontrada.                    |
| 409    | Certificado já emitido para a inscrição.     |
| 422    | Inscrição sem presença confirmada.           |

## 1.8 Visualização/Download do Certificado

GET /api/v1/certificates/{certificateId}/download

- **Descrição:** Retorna o arquivo PDF do certificado. Apenas o titular, organizador do evento ou admin podem acessar.
- **Autenticação:** requer token JWT.

**Entrada:** nenhum corpo.

**Saída:** 200 OK com o arquivo `application/pdf`. Não há corpo JSON na resposta.

| Código | Significado                            |
|--------|----------------------------------------|
| 200    | Arquivo enviado com sucesso.           |
| 401    | Usuário não autenticado.               |
| 403    | Usuário não tem permissão.             |
| 404    | Certificado ou arquivo não encontrado. |

## 1.9 Listagem de Usuários (Admin)

GET /api/v1/users

- **Descrição:** Lista todos os usuários cadastrados. Apenas administradores podem acessar.
- **Autenticação:** requer token JWT.

- **Parâmetro opcional:** `?role=ORGANIZER, ADMIN` ou `PARTICIPANT`.

**Entrada:** nenhum corpo.

**Saída (JSON):** 200 OK

```
[
  {
    "id": 1,
    "firstName": "João",
    "lastName": "Silva",
    "email": "joao@email.com",
    "role": "PARTICIPANT",
    "createdAt": "2026-09-01T10:00:00"
  },
  {
    "id": 2,
    "firstName": "Maria",
    "lastName": "Souza",
    "email": "maria@email.com",
    "role": "ORGANIZER",
    "createdAt": "2026-09-01T11:00:00"
  }
]
```

| Código | Significado                       |
|--------|-----------------------------------|
| 200    | Lista retornada com sucesso.      |
| 400    | Papel usado no filtro é inválido. |
| 401    | Usuário não autenticado.          |
| 403    | Usuário não é administrador.      |

## 1.10 Atualização de Papel (Admin)

**PATCH** `/api/v1/users/{userId}/role`

- **Descrição:** Altera o papel de usuário. Apenas administradores podem acessar.
- **Autenticação:** requer token JWT.

**Entrada (JSON)**

```
{
  "role": "ORGANIZER"
}
```

**Saída (JSON):** 200 OK

```
{
  "id": 1,
  "firstName": "João",
  "lastName": "Silva",
  "email": "joao@email.com",
  "role": "ORGANIZER",
  "createdAt": "2026-09-01T10:00:00"
}
```

| Código | Significado                   |
|--------|-------------------------------|
| 200    | Papel atualizado com sucesso. |
| 400    | Papel inválido.               |
| 401    | Usuário não autenticado.      |
| 403    | Usuário não é administrador.  |
| 404    | Usuário não encontrado.       |

## 1.11 Edição de Evento

PATCH /api/v1/events/{eventId}

- **Descrição:** Atualiza parcialmente um evento ativo. Apenas organizador proprietário ou admin podem acessar; `organizerId` não pode ser alterado.
- **Autenticação:** requer token JWT.

### Entrada (JSON) — exemplo

```
{
  "capacity": 70,
  "location": "Auditório Principal"
}
```

Saída (JSON): 200 OK

```
{
  "id": 6,
  "title": "Aprenda GIT de uma vez por todas!",
  "location": "Auditório Principal",
  "capacity": 70,
  "status": "ACTIVE",
  "organizerId": 2
}
```

| Código | Significado                                        |
|--------|----------------------------------------------------|
| 200    | Evento atualizado com sucesso.                     |
| 400    | Campo enviado é inválido.                          |
| 401    | Usuário não autenticado.                           |
| 403    | Usuário não é proprietário nem admin.              |
| 404    | Evento não encontrado.                             |
| 409    | Evento cancelado/finalizado não pode ser editado.  |
| 422    | Regra de negócio, como data inválida, foi violada. |

## 1.12 Cancelamento de Evento

PATCH /api/v1/events/{eventId}/cancel

- **Descrição:** Cancela evento ativo. Apenas organizador proprietário ou admin podem acessar. Eventos cancelados não recebem novas inscrições.
- **Autenticação:** requer token JWT.

**Entrada:** nenhum corpo.

**Saída (JSON):** 200 OK

```
{
  "id": 6,
  "title": "Aprenda GIT de uma vez por todas!",
  "status": "CANCELED",
  "organizerId": 2
}
```

| Código | Significado                           |
|--------|---------------------------------------|
| 200    | Evento cancelado com sucesso.         |
| 401    | Usuário não autenticado.              |
| 403    | Usuário não é proprietário nem admin. |
| 404    | Evento não encontrado.                |
| 409    | Evento já cancelado ou finalizado.    |

## 1.13 Cancelamento de Inscrição

PATCH /api/v1/registrations/{registrationId}/cancel

- **Descrição:** Cancela inscrição ativa antes da ocorrência do evento. Somente o titular da inscrição pode acessar.
- **Autenticação:** requer token JWT.

**Entrada:** nenhum corpo.

**Saída (JSON):** 200 OK

```
{
  "id": 10,
  "userId": 1,
  "eventId": 1,
  "status": "CANCELED",
  "attendanceConfirmed": false,
  "attendanceConfirmedAt": null
}
```

| Código | Significado                                    |
|--------|------------------------------------------------|
| 200    | Inscrição cancelada com sucesso.               |
| 401    | Usuário não autenticado.                       |
| 403    | Inscrição não pertence ao usuário autenticado. |
| 404    | Inscrição não encontrada.                      |
| 409    | Inscrição já está cancelada ou atendida.       |
| 422    | Evento já ocorreu.                             |

## 1.14 Minhas Inscrições

GET /api/v1/users/me/registrations

- **Descrição:** Lista inscrições do usuário autenticado, com um resumo do evento associado.
- **Autenticação:** requer token JWT.

**Entrada:** nenhum corpo.

**Saída (JSON):** 200 OK

```
[
  {
    "id": 10,
    "eventId": 1,
    "status": "ACTIVE",
    "registrationAt": "2026-09-01T10:30:00",
    "event": {
      "title": "QA na prática",
      "date": "2026-10-10",

```

```
    "hour": "08:30",  
    "status": "ACTIVE"  
  }  
}  
]
```

| Código | Significado |
|--------|-------------|
|--------|-------------|

|     |                                                            |
|-----|------------------------------------------------------------|
| 200 | Inscrições retornadas com sucesso; a lista pode ser vazia. |
|-----|------------------------------------------------------------|

|     |                          |
|-----|--------------------------|
| 401 | Usuário não autenticado. |
|-----|--------------------------|

## Arquitetura do ValidAI

### Decisão

O ValidAI adota uma arquitetura em camadas, organizada em quatro áreas: **api**, **domain**, **core** e **infra**. É a alternativa mais adequada para uma API Spring Boot de porte acadêmico: separa responsabilidades, segue convenções do framework e permite testar regras de negócio sem depender de HTTP ou PostgreSQL.

```
src/main/java/com/yvital/validai/  
├── api/  
│   ├── controller/  
│   ├── dto/request/  
│   ├── dto/response/  
│   └── mapper/  
├── domain/  
│   ├── model/  
│   ├── enums/  
│   ├── repository/  
│   └── service/  
├── core/  
│   ├── config/  
│   ├── exception/  
│   └── security/  
└── infra/  
    ├── database/  
    ├── qrcode/  
    └── storage/
```

```
Cliente → Controller → Service → Repository → PostgreSQL  
                        ↓  
                    QR Code / PDF
```

### Responsabilidades

| Camada | Faz                                                                       | Não faz                                                  |
|--------|---------------------------------------------------------------------------|----------------------------------------------------------|
| api    | Traduz HTTP, valida DTOs, chama serviços e produz respostas.              | Não contém regra de capacidade, permissão ou status.     |
| domain | Representa entidades, enums, repositórios e regras de negócio.            | Não conhece detalhes de cabeçalhos HTTP ou formato JSON. |
| core   | Centraliza configurações, tratamento global de erros e, futuramente, JWT. | Não implementa casos de uso específicos.                 |
| infra  | Encapsula detalhes técnicos de banco, token QR, arquivos PDF e storage.   | Não decide se um certificado pode ser emitido.           |

## Fundamentos da escolha

- **Separação de responsabilidades:** cada mudança tem lugar claro; uma alteração de JSON não exige mudar regra de inscrição.
- **Baixo acoplamento:** controller depende de serviço, e serviço depende de repositório; nenhuma camada inferior chama uma superior.
- **Testabilidade:** capacidade, duplicidade e emissão única de certificado podem ser testadas no serviço com dependências simuladas.
- **Manutenibilidade:** exceções, validações e configurações não são duplicadas entre endpoints.
- **Aderência ao Spring:** controllers, services, repositories e JPA são conceitos ensinados e amplamente usados no ecossistema.

## Decisões técnicas

| Tema                  | Decisão                                                                                                                                |
|-----------------------|----------------------------------------------------------------------------------------------------------------------------------------|
| Linguagem e framework | Java 21 e Spring Boot 4.1.0                                                                                                            |
| Persistência          | Spring Data JPA e PostgreSQL 16.                                                                                                       |
| Schema                | Flyway; cada alteração estrutural é uma migration SQL imutável e versionada.                                                           |
| Validação             | Bean Validation nos DTOs; regras que consultam banco pertencem aos serviços.                                                           |
| Erros                 | Handler global retorna <b>timestamp</b> , <b>status</b> , <b>code</b> , <b>message</b> , <b>path</b> e <b>fields</b> quando aplicável. |
| QR Code               | Token opaco, aleatório, único por inscrição. A API valida o conteúdo lido; não implementa câmera.                                      |
| Certificado           | Registro único por inscrição; PDF em diretório local configurável, fora do Git.                                                        |
| Segurança             | JWT e autorização efetiva depois da 1AV; a matriz de permissões já define o comportamento alvo.                                        |
| Documentação          | OpenAPI/Swagger para a API implementada.                                                                                               |